# ADVANCED OBJECT-ORIENTED PROGRAMMING USING JAVA

## CASE STUDY 3: BANK ACCOUNT AND MONEY TRANSFERS

**Problem Statement:**

A bank needs a Java app for accounts, deposits, withdrawals, and transfers. Its current design (public fields, primitives, separate service methods) causes these problems: overdraft limits exceeded, money lost when a transfer fails midway, frozen accounts allowing withdrawals, balances not matching history, and zero or negative transactions. Task: analyse the problems and propose an improved object-oriented domain model.

### Q1. Identification of Domain Invariants (6 Marks)

Identify at least six business invariants that must be maintained in the banking application. For each invariant:

- State the business rule clearly.
- Explain why the rule is important.
- Identify the possible consequences if the rule is violated.

| Business Rule | Why Important | Consequence if Violated |
|---|---|---|
| Transaction amount must be greater than zero. | Zero or negative amounts have no business meaning; a negative deposit is a hidden withdrawal. | Fraud loopholes, bypassed checks, corrupted history. |
| Balance must never go below −(overdraft limit). | The limit controls the bank's credit risk. | Unapproved loans, financial loss, regulatory trouble. |
| A frozen account cannot be debited (withdrawals or outgoing transfers). | Freezes are due to fraud, legal orders, or KYC issues. | Legal and compliance breach, funds leaving a disputed account. |
| A closed account cannot accept any transaction, and can be closed only when balance is zero. | Closing ends the account's lifecycle. | Money stuck in dead accounts, customer disputes. |
| A transfer is atomic: debit and credit both happen or neither does (total money is conserved). | Money cannot vanish or be created. | Customer loses money, books out of balance, loss of trust. |
| Balance must equal opening balance plus the sum of all ledger entries. | The ledger is the audit trail and source of truth. | Failed audits, undetected fraud, unresolvable disputes. |

**Q2. Entities and Value Objects (6 Marks)**

Identify the important domain concepts involved in the banking application. Classify the following concepts as entities or value objects:

- Bank Account
- Account Number
- Money
- Ledger Entry
- Overdraft Limit
- Fund Transfer

Justify your classification based on identity, equality, lifecycle and mutability.

- **Entity:** has a unique ID, equality by ID, has a lifecycle, mutable.
- **Value Object:** no ID, equality by value, immutable, replaced instead of changed.

| Concept | Type | Identity | Equality | Lifecycle | Mutability |
|---|---|---|---|---|---|
| **Bank Account** | Entity | Unique account number | By ID: two accounts with the same balance are still different | Active → Frozen → Closed | Mutable (balance, status change) |
| **Account Number** | Value Object | None of its own (it *is* the ID of the account) | By value: same digits means same number | None | Immutable |
| **Money** | Value Object | None | By value: ₹500 = any other ₹500 (amount + currency) | None | Immutable: plus() and minus() return a new Money |
| **Ledger Entry** | Entity (immutable once created) | Unique entry ID, traceable for audit | By ID: two ₹500 deposits are still separate entries | Created once, never modified or deleted | Immutable after creation |
| **Overdraft Limit** | Value Object | None | By value: same amount means same limit | None | Immutable: to change it, replace with a new limit |
| **Fund Transfer** | Entity | Unique transfer ID | By ID: two ₹5,000 transfers between the same accounts are different transfers | INITIATED → COMPLETED / FAILED / REVERSED | Mutable (status changes) |

- Account Number is a value object even though it identifies an account. It is an *identifier*, not an entity.
- Ledger Entry is debatable, since it is immutable like a value object. It is an entity because each entry has a unique ID and must be individually traceable.
- Overdraft Limit is just a `Money` value with a rule (cannot be negative).